Part 1: Understanding Consensus (Happy Path)

In the happy_path.js scenario, consensus is achieved easily since all nodes are operational and follow the classical two-phase protocol. The proposer kicks o Phase 1 by broadcasting a Prepare message tagged with a unique, monotonically increasing proposal number. Since there are no competing rounds, all three acceptors reply with a Promise, committing not to acknowledge older proposals and reporting no prior values.

Once the proposer gets a majority (a quorum of 2 out of 3 acceptors) to answer their Promise, it goes to Phase 2 where it broadcasts an Accept request with its proposed value. The acceptors still nd the proposal number valid against their promises, accept the value, and record it locally. The moment a majority records the acceptance, the value is o icially chosen.

Finally, learners get an update with the outcome and update their state.

Consensus is achieved since the system followed a coordinated path through both phases without any contention or node drops. The learners don't have a vote and just record the officially decided choice. This establishes the baseline mental model that agreement doesn't depend on a single dictator, but synchronized quorum formation across independent and cooperating nodes.

Part 2: Acceptor Failure

The acceptor_failure.js scenario demonstrates the main fault-tolerance property of Paxos: a decision only needs majority vote, not unanimity. In a three-acceptor cluster, the minimum quorum size is [n/2] + 1 = 2. When one acceptor crashes and goes o line, two acceptors remain active.

During Phase 1, the proposer emits its Prepare message to all nodes. Although the crashed acceptor is silent, the two active acceptors reply with valid promises satisfying the quorum threshold. The proposer moves directly to Phase 2, sending out Accept requests. Both acceptors process and accept the proposal, establishing a valid majority acceptance. Since two nodes agree, the system considers the value chosen and notifies learners.

Consensus is achieved uninterrupted since Paxos assumes honest, crash-stop participants, not faultless infrastructure. The protocol easily tolerates f failures in a system of 2f + 1 nodes. As long as at least f + 1 nodes participate, progress is guaranteed. No unanimity is

needed; overlapping majority su ce to provide collective memory to guarantee safety and operational liveness.

Part 3: Competing Proposers

The competing_proposers.js scenario shows what happens when two proposers try to lead consensus simultaneously. In Paxos, proposers avoid deadlock by using proposal numbers to establish a strict temporal order. These numbers do not say one value is better than another; they simply determine which proposer has priority to proceed.

Here, Proposer A issues a Prepare with a lower identi er and Proposer B simultaneously issues a Prepare with a higher identi er. When acceptors receive B's higher number, they promise never to accept any future request carrying a lower identi er. When Proposer A tries to submit its Accept message (or outdated Prepare), the acceptors reject it as its number violates their newer promises. Proposer B secures the majority promise and drives its value to acceptance, pushing Proposer A aside without malicious intent. Proposers must honor their highest promise, and proposal numbers act as preemptive locks. Without them, competing proposers could endlessly overwrite one another, destroying consistency and stalling agreement.

Part 4: Partial Acceptance

The partial_acceptance.js scenario explores how Paxos safeguards in- ight progress when a proposer fails mid-stream. In this experiment, Proposer 1 submits an Accept message for value V_1, but it is accepted by only one acceptor before Proposer 1 stalls or loses connectivity. This single acceptance is insu cient to choose V_1, leaving the system in an ambiguous partially accepted state.

When Proposer 2 comes in with a higher proposal number and its chosen value V_2, it starts Phase 1 by broadcasting Prepare. Crucially, the acceptor that accepted V_1 reports this prior acceptance back in its promise response. Paxos rules strictly require Proposer 2 to inspect all promise responses and adopt the value associated with the highest proposal number reported by any acceptor in the quorum. Consequently, Proposer 2 drops its intended value V_2, adopting V_1 for its Phase 2 Accept round. It then drives V_1 across a majority. This prevents a dual-state split: proposers may change, but any value that might

have been chosen or partially accepted must be adopted and driven to preserve historical continuity.

Part 5: Preserving Chosen Values

In preserve_chosen_value.js, a value V_1 has achieved majority acceptance in an earlier round, and a new proposer comes online with a completely di erent agenda, trying to establish value V_2. This task demonstrates that once Paxos chooses a decision, that decision becomes permanent and cannot be overwritten.

The new proposer generates a higher proposal number and initiates Phase 1 across the acceptors. Since any majority quorum must intersect with the previous winning quorum by at least one node (the Pigeonhole Principle), at least one acceptor contacted by the new proposer is guaranteed to hold the record of V_1.

In its Promise reply, that overlapping acceptor alerts the new proposer that V_1 was already accepted under an earlier round. Adhering to the core Paxos invariant, the new proposer must discard its own candidate value V_2 and use V_1 in its Accept phase. The learners once again learn V_1. This demonstrates that Paxos protects decisions, not proposers. Even when proposers are blind to past events, the overlapping memory of the quorum ensures that established history cannot be rewritten.

Part 6: Multi-Paxos and Stable Proposers

The multi_paxos.js scenario demonstrates how consensus overhead in real-world deployments can be minimized. Basic Paxos requires two full round-trips for every decision: Phase 1 (Prepare/Promise) and Phase 2 (Accept/Accepted). When processing continuous streams of log entries, repeatedly running Phase 1 becomes a major bottleneck.

Multi-Paxos amortizes Phase 1 by letting a proposer successfully issue Prepare messages that cover multiple consecutive log slots, and acceptors grant an extended promise.

Once this stable proposer emerges, it acts as a de facto leader. For subsequent decisions, it skips Phase 1, directly sending Phase 2 Accept requests.

The simulator output shows sequenced decisions reaching consensus in a single round-trip instead of two. There's no complicated out-of-band leader election algorithm; leadership simply emerges as long as other nodes refrain from sending higher proposal

numbers. This optimization drastically reduces message exchange and latency while retaining safety. If the stable proposer dies, another node can re-issue a Phase 1 Prepare with a higher number to assume the role.

Part 7.1: Network Partitioning

In network_partition.js, communication links between nodes are severed, splitting the network into isolated segments. This test demonstrates how Paxos behaves when messages are dropped or delayed, illustrating the trade-o between consistency and availability formalized by the CAP theorem.

When the cluster splits into a minority partition (e.g., 1 node) and a majority partition (e.g., 2 nodes in a 3-node system), nodes on the minority side cannot form a quorum. If a proposer isolated on the minority side attempts to broadcast Prepare messages, its requests cannot gather su icient promises. It hangs or fails, unable to make progress. Conversely, proposers operating on the majority component complete rounds normally.

The partition impacts liveness (availability), but safety (consistency) is untouched.

Paxos never produces con icting decisions across partitions since they are impossible for two disjoint network segments to both hold a majority of the same con guration. Paxos deliberately chooses safety over availability: it will freeze rather than risk recording con icting decisions.

Part 7.2: Majority Loss

The majority_loss.js scenario demonstrates what happens when failure thresholds exceed tolerable limits. In a 3-node system, the protocol requires agreement from at least 2 acceptors. When 2 acceptors fail or become completely detached, only 1 operational acceptor remains.

The proposer attempts to start consensus by broadcasting Prepare messages. It receives a promise from the single surviving acceptor, but its quorum calculation logic determines that 1 out of 3 is strictly less than the required threshold of 2. Since the quorum requirement cannot be satis ed, the consensus engine cannot progress to Phase 2. The protocol completely stalls.

This demonstrates the limit of Paxos fault tolerance ($f < n/2$). While the system loses liveness and cannot serve write requests, safety is preserved: no illegal, partial, or split-

brain decisions are committed to learners. Paxos safely waits until network connectivity or failed nodes are restored before resuming. Progress is held hostage to quorum availability.

Part 7.3: State Loss

In state_loss.js, an acceptor crashes and subsequently reboots, but its local storage is volatile. Upon rebooting, its recorded promises and previously accepted values are reset to zero or null. This scenario demonstrates that Paxos requires persistent storage to maintain safety guarantees.

Suppose an acceptor accepted value V_1 before rebooting. After rebooting with wiped memory, a new proposer comes along with a higher proposal number and asks for a promise. Instead of reporting that it previously accepted V_1, the amnesic acceptor reports a blank slate. Because of this missing history, the new proposer mistakenly believes it is free to push a con icting value V_2.

If other nodes also experience state loss or are unreachable, two con icting values can be accepted across di erent rounds, causing an irrecoverable split-brain condition.

Paxos mathematically relies on the premise that an acceptor's promise is permanent, and its memory spans reboots. Without write-ahead logging to disk (fsync), crashes violate foundational assumptions and destroy consistency.

Part 7.4: Quorum Misconfiguration

The quorum_misconfiguration.js scenario demonstrates the mathematical foundation of Paxos: quorums must always overlap. In this exercise, the quorum threshold is miscon gured, for instance, relaxed to 1 out of 3, or set to independent sub-groups of 2 in a 4-node cluster without a strict majority requirement (Q <=n/2).

With the quorum requirement weakened, Proposer A gathers a "quorum" from Acceptor 1 and commits its value V_A. Concurrently, Proposer B gathers a separate "quorum" from Acceptor 2 and commits value V_B. Since the two quorums share no common acceptor, neither proposer learns about the other's proposal. Acceptors issue promises independently.

As a result, learners observe two contradictory values chosen for the same slot.

This is a catastrophic breakdown of consensus safety. The scenario demonstrates that quorum sizing is a strict security invariant, not an arbitrary performance tuning parameter.

A quorum must always strictly satisfy Q >= n/2 + 1 so that any two quorums have at least one node in common to enforce value inheritance.